

Development and Explanation

Overview

The development of **DarshanEase** follows a modular approach using the **MERN Stack** (**M**ongoDB, **E**xpress.js, **R**ect.js, and **N**ode.js) along with the **MVC (Model–View–Controller)** architecture. The application is divided into frontend and backend modules that communicate through RESTful APIs. This architecture ensures scalability, maintainability, and secure data management.

Phase 1: Project Initialization

The development process begins by creating separate folders for the frontend and backend applications.

Frontend

- React.js project created using Vite.
- TypeScript configured for type safety.
- Tailwind CSS integrated for responsive UI design.

Backend

- Node.js project initialized.
- Express.js installed for API development.
- MongoDB connected using Mongoose.

Phase 2: Database Development

MongoDB is used as the database to store all application data.

Collections

Users

Stores user account information.

Fields:

- User ID

- Name
- Email
- Phone Number
- Password
- Role

Temples

Stores temple information.

Fields:

- Temple ID
- Temple Name
- Location
- Description
- Opening Time
- Closing Time

Darshan Slots

Stores available darshan schedules.

Fields:

- Slot ID
- Temple ID
- Date
- Start Time
- End Time
- Available Seats

Bookings

Stores booking records.

Fields:

- Booking ID

- User ID
- Slot ID
- Booking Date
- Booking Status
- Number of Devotees

Phase 3: Backend Development

The backend handles business logic and API requests.

Authentication Module

Functions:

- User Registration
- User Login
- Password Encryption
- JWT Authentication

Temple Module

Functions:

- Add Temple
- Update Temple
- Delete Temple
- View Temple Details

Darshan Slot Module

Functions:

- Create Slot
- Update Slot
- Delete Slot
- Display Available Slots

Booking Module

Functions:

- Book Ticket
- View Booking History
- Cancel Booking
- Validate Slot Availability

Admin Module

Functions:

- Manage Users
- Manage Bookings
- Generate Reports

Phase 4: Frontend Development

The frontend provides an interactive interface for users.

Pages Developed

- Home Page
- Login Page
- Registration Page
- Temple Listing Page
- Temple Details Page
- Darshan Slot Selection Page
- Booking Confirmation Page
- Booking History Page
- User Profile Page
- Admin Dashboard

Components Developed

- Navigation Bar
- Footer
- Temple Card
- Booking Form
- Slot Card
- Login Form
- Registration Form
- Loading Spinner
- Alert Messages

Phase 5: API Development

RESTful APIs are developed to enable communication between the frontend and backend.

Authentication APIs

- POST /api/auth/register
- POST /api/auth/login

Temple APIs

- GET /api/temples
- POST /api/temples
- PUT /api/temples/:id
- DELETE /api/temples/:id

Slot APIs

- GET /api/slots
- POST /api/slots

Booking APIs

- POST /api/bookings
- GET /api/bookings

- DELETE /api/bookings/:id

Phase 6: Testing

The application is tested to ensure functionality and reliability.

Unit Testing

Tests individual modules such as authentication, booking, and slot management.

Integration Testing

Verifies communication between frontend, backend, and database.

User Interface Testing

Checks responsiveness, navigation, and usability across devices.

Functional Testing

Ensures all features work according to the requirements.

Phase 7: Deployment

The application is prepared for deployment.

Frontend

- Build using Vite.
- Deploy on platforms such as Vercel or Netlify.

Backend

- Deploy on platforms such as Render or Railway.

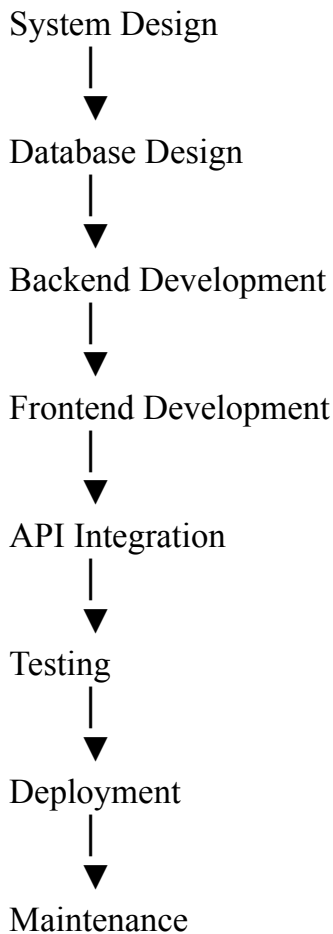
Database

- Host MongoDB using MongoDB Atlas.

Development Workflow

Requirement Analysis





Explanation of the Workflow

1. **Requirement Analysis:** Identify user needs and define the project scope.
2. **System Design:** Design the architecture, database, and user interface.
3. **Database Design:** Create MongoDB collections and define relationships.
4. **Backend Development:** Implement business logic, authentication, and REST APIs.
5. **Frontend Development:** Build responsive user interfaces with React and Tailwind CSS.
6. **API Integration:** Connect frontend components to backend services using Axios.
7. **Testing:** Validate functionality, security, and performance.
8. **Deployment:** Host the application and database for public access.
9. **Maintenance:** Monitor, fix issues, and add new features over time.

Advantages of the Development Approach

- Modular architecture for easy maintenance.

- Clear separation of frontend and backend.
- Secure authentication using JWT.
- Scalable REST API design.
- Efficient database management with MongoDB.
- Responsive and user-friendly interface.
- Easy integration of future enhancements such as online payments, QR code verification, notifications, and mobile applications.

Conclusion

The development of DarshanEase follows modern software engineering practices by combining the MERN stack with the MVC architecture. The application is built in well-defined phases, from planning and design to implementation, testing, and deployment. This structured approach results in a secure, scalable, and maintainable temple darshan ticket booking system that delivers a seamless booking experience for devotees while providing efficient management tools for administrators.